MyPlace

Crashing Out

Alex DiMichele, Chris Ross, Sebastian Tran, Logan Yates, Ali Abdulrazzak

Chapter 1: Team Vision

Overview

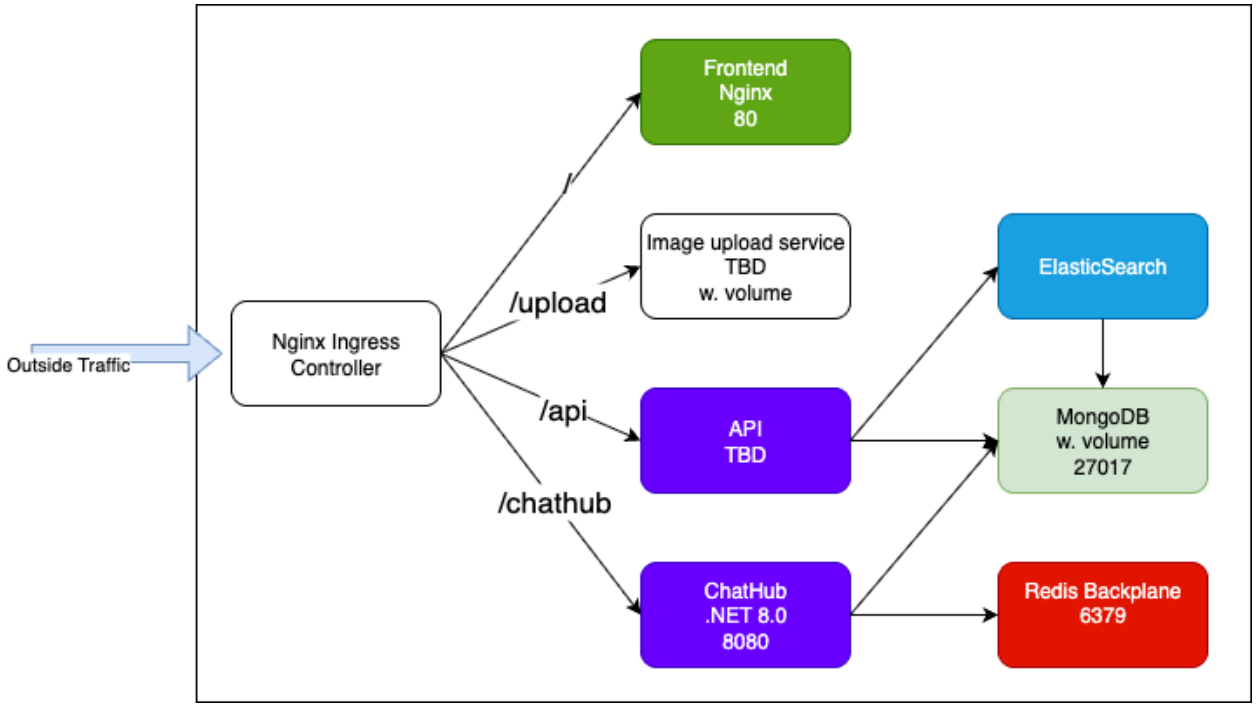
Roommate finding can be an egregious process.

Different living styles, tolerances for cleanliness, and acceptable noise levels are all factors that can lead to conflict among roommates.

The main goal of our app is to streamline the process of finding roommates by providing an easy to use recommendation system and messaging platform. By providing a recommendation system that puts living style needs first, we hope to make finding a roommate a much less stressful endeavor.

After completing our recommendation system, we hope to also provide a platform for owners to post listings for rental properties. By implementing both, we hope to have an end to end application where users can find roommates then easily find a place to rent.

Architecture Design



Nginx Ingress Controller:
The Nginx Ingress Controller will be responsible for routing traffic to the correct deployment. By including an ingress, each service associated with a deployment can be a simple ClusterIP as well. The reason for

including an ingress will be further elaborated on in chapter 2, but briefly, websocket traffic requires an ingress.

Frontend
Our frontend will be served via Nginx. This will be the endpoint for our interface.

API
The API will be responsible for creating profiles/logging in, keeping track of matches, and providing recommendations.

ChatHub
Our ChatHub will be responsible for handling the websocket connections to enable messaging in our application.

Redis Backplane
The Redis backplane will allow for sharing connection information between ChatHub pods. This enables sending messages between what would have been independent pods. In addition, we can also use redis as a key-value storage for storing routing information.

Image Upload Service
Images will be an important part of our app, as all profiles will require images that other users will view. The image upload service will handle this operation.

MongoDB (w. Persistent volume)
MongoDb will be where all profiles, messages, and picture locations will be stored.

ElasticSearch
ElasticSearch will be an interface for our API to provide curated roommate recommendations.

User Data Flow / Requirements

Registering a User:
As a user, I want to be able to register an account so I can create a profile.

- Given I am a new user on the registration page, when I enter a valid email address and password, then I should be redirected to profile creation.
- User inputs email address → API checks if it is unused → Continue with profile creation.

Logging In:
As a user, I want to securely log into my account so I can access my profile and matches.

- Given I am on the login page, when I provide valid credentials, then I should be redirected to my dashboard.
- User inputs credentials → API verifies against MongoDB → Return authentication token → Redirect to dashboard.

Profile Creation:
As a user, I want to add details to my profile so I can share information about myself.

- Given I am on the profile creation page, when I upload a valid image, then it should be saved and displayed on my profile.

- User uploads file → Upload service saves image and returns link to image → User can view image on profile.

- Given I have filled out all required information, when I save my profile, then I should be able to begin matching.

- User requests to save profile → Check information completeness → API saves to MongoDB → Redirect to matching.

Finding Matches / Recommendations:
As a user, I want to view suggested roommates so I can match with them.

- Given I am logged in and on the roommate suggestion page, when I request to match with another user, then the other user should be alerted.
- User swipes to match → API records match request → ElasticSearch generates updated suggestions → API adds match request to other user's queue.

Searching for Users or Properties:
As a user, I want to search for roommates or property listings so I can find the best match for my needs.

- Given I enter search criteria, when I submit the request, then I should see relevant results.
- User enters criteria → API queries ElasticSearch → ElasticSearch returns ranked results → Results displayed in frontend.

Messaging:
As a user, I want to be able to message a match so I can determine if we're actually compatible.

- Given the other user has accepted our match request, when I send a message to them, then it should be sent and saved.
- User sends a message → ChatHub routes via Redis backplane → Message stored in MongoDB → Recipient receives message.

As a user, I want to create a group chat so I can have all my roommates in one place.

- Given I have matched with multiple users, when I create a chat with one user, then I should have the option to add more to the chat.

Image Retrieval:
As a user, I want profile images to load quickly so I can evaluate potential matches.

- Given I am browsing matches, when profiles are displayed, then I should see their profile pictures.
- Frontend requests image → API checks Image ID → Image retrieved from persistent storage via Upload Service → Returned to frontend for display.

Error Handling & Notifications:
As a user, I want to be notified if something goes wrong so I know what action to take.

- Given I lose connection during a chat, when I attempt to send a message, then I should see a notification.
- ChatHub detects failed delivery → Sends error event → Frontend displays notification.

Team Charter

Sebastian Tran – ChatHub server
Chris Ross – Frontend
Ali, Alex DiMichele, Logan Yates – API & Image Upload Service

Meet as needed, keep updated over Discord regularly

Chapter 2: Implementation details

In this chapter, we cover the implementation details of our architecture along with design considerations we may need to consider.

ChatHub
Part of ASP.NET Core's core libraries is SignalR, a websocket interface for C#. Websockets will be needed to send and receive messages in our chat component. If messages were served via a REST API, the client would need to regularly poll the server to check for a new message. With websockets however, the connection is kept open, and the client will know when a new message has been sent.

SignalR provides the option to use a Redis backplane as well as easy to use integration with authentication services, which makes ASP.net Core a top tier choice for the websocket server. However, websockets do add extra complexity to the frontend in integration with the chat hub. Extra coordination is needed to provide an interface for both the client and server to communicate with each other.

Nginx Ingress Controller:
While all traffic could be routed through the frontend Nginx deployment, the inclusion of websockets for our chat component unfortunately complicates the reality.

Websockets require a persistent connection to the server. However, if multiple server pods exist, the frontend Nginx service can route to any of them, breaking the persistent connection.

A higher level networking service is required to effectively route websocket traffic, in this case, the Nginx Ingress Controller. The Nginx Frontend deployment is now only responsible for serving our webpage. We configure a sticky connection for the controller, which ensures that a user with a websocket connection will only make connections to the same pod. The persistent connection is now not broken.

Furthermore, the Nginx Ingress Controller simplifies routing. Each pod that would've been exposed outside of the cluster with multiple services, now can be easily routed to by using one controller.

Redis backplane
Another issue that arises when using websockets with multiple server pods is sharing information. For instance, if one client sends a message to another client, the server may not know how to route the connection. With one server pod, the server knows all the connected clients and can successfully send the message. However, with multiple pods, different clients can be connected to different pods, each of which is unaware of the other connection. In this scenario, the message cannot be routed.

The issue here is solved via the use of a Redis Backplane. Both ChatHub pods connect to a Redis deployment to share their connection data. Through the inclusion of a Redis backplane, both server pods know about each other's connections and can successfully route the message.

While it is possible to use just a Redis backplane alone, there is a noticeable delay updating and establishing connections on one pod then making them known across the deployment. Thus, the inclusion of both the Nginx Ingress Controller and the Redis backplane are needed to manage websocket connection seamlessly.

We can also use Redis as a database for routing. The client needs to know who to send a message to. While this information can be stored on the browser, consolidating the information into Redis and ChatHub server will make frontend development easier.

ElasticSearch
A core component of the application will be providing the user with recommendations. To provide curated recommendations, ElasticSearch will be used. We have no experience with ElasticSearch, but several libraries are available to integrate it with our API.

API
The API will be responsible for handling the asynchronous tasks of our backend. As we plan, the API will integrate with elastic search to provide recommendations, issue JWTs and refresh tokens for login services, and save profiles to MongoDb. However, it is possible to split each aspect into its own deployment to separate concerns. Many web frameworks exist to easily create an API, but given our group skillsets, we believe either ASP.net Core or Spring boot is viable.

Image Upload Service:
We will have a dedicated deployment for image uploads. The image upload service will handle the storing and retrieving of images. When a user uploads an image, the upload service will save it to a persistent volume. Then, it will store the image path in the persistent volume, and an associated ID in the database. When the application needs these images, the application will request the images via the associated ID.

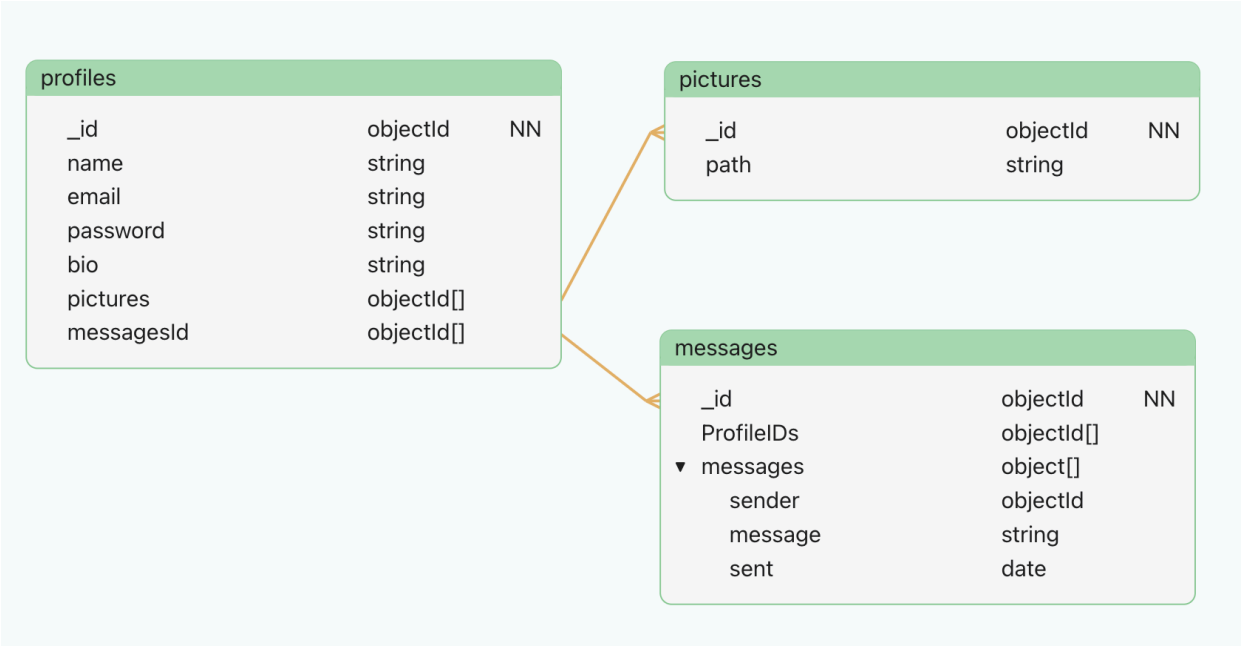
Frontend/Nginx
Nginx will serve our frontend. We plan to develop our frontend with React + Typescript. Typescript makes us less error prone to passing faulty data. Interactivity is implemented with React. We can also extend our app easily with NPM's large library of packages. Anime.js and WebGL could be worth looking into if time allows.

MongoDb
MongoDb will serve as our storage for profile information, messages, and image file locations.

While a relational database like PostgreSQL can be used, our information is largely grouped together with little need for joins across tables. We believe this makes a non-relational database like MongoDb a better choice for us. In addition, MongoDb also has support for geographic information, which we may utilize for matching.

Unfortunately though, we will need to implement our own user models for storing emails and passwords with MongoDb. If we choose a relational database, we could use EFCore, an excellent object relational mapper. EFCore provides user models that are easy to extend, while including safety for keeping sensitive information safe.

We will also need to provide MongoDb with a persistent volume in order to store information even after the deployment is removed.


profiles
_id objectId NN
name string
email string
password string
bio string
pictures objectId[]
messagesId objectId[]
pictures
_id objectId NN
path string
messages
_id objectId NN
ProfileIDs objectId[]
▼ messages object[]
sender objectId
message string
sent date

Team Resumes

Christian Ross

215-667-5793 **christian.kross@outlook.com**

EDUCATION

West Chester University of Pennsylvania | *West Chester, PA*

B.S. in Computer Science | *August 2024 – present* | *Expected Graduation: May 2026*

Delaware County Community College | *Media, PA*

A.S. in Computer Science | *August 2021 – May 2024*

WORK EXPERIENCE

Chipotle Mexican Grill | *Various Locations, PA*

Service/Kitchen Manager | *May 2022 – present*

- Led and trained teams of employees to deliver exceptional customer service and improve operational efficiency.
- Oversaw daily operations, including inventory management and compliance with health and safety standards.
- Collaborated with upper management to implement process improvements, reducing waste and increasing efficiency.

PROJECTS

Aggre-Gator RSS

- Collaborated with a four-person team to develop an RSS/Atom feed aggregation web application, implementing frontend functionality in TypeScript with modular UI components and client-side logic.
- Contributed to frontend–backend integration by implementing RESTful API endpoints on the client side and coordinating data flow between UI components and server interfaces.
- Implemented Google OAuth authentication system to enable secure user login and session management.

Air-Traffic Control Simulator

- Designed a Java-based simulation that models air traffic patterns, managing multiple aircraft and simulating real-time control scenarios with the goal of safe and efficient airspace coordination.
- Generated comprehensive flight statistics based on simulation outcomes including runway utilization efficiency, aircraft wait times, and airspace coordination metrics to analyze and optimize system performance.

RELEVANT COURSEWORK & EXPERIENCE

- Foundations of Computer Science
- Computer Science I, II, & III
- Data Structures and Algorithms
- Computer Systems
- Software Security

- HTML/CSS
- JavaScript/TypeScript
- Proficient in programming with Java
- Proficient using Windows & macOS
- Experience using Linux
- Experience working with Git, GitHub, GitHub Actions

AWARDS & ACHIEVEMENTS

- Dean's List Fall 2024, Spring 2024, Fall 2022
- GPA: 3.8

Sebastian Tran

transebastian91@gmail.com - 215 284 9916 - sebastiantran.azurewebsites.net - github.com/stran9682

Education

West Chester University of Pennsylvania, GPA: 3.98 *August 2023 - May 2026*

Accelerated MS + BS Computer Science, Minor in Applied Statistics West Chester, PA

Experience

Department Tutor – West Chester University *August 2025 – present*

- Providing tutoring services to students in introductory to advanced level computer science courses
- Work through study and review content for topic requested by student individually and providing feedback

Undergrad Research Assistant – West Chester University *July 2025 – present*

- Working with professor and other research assistants in developing university video sharing platform
- In-progress work of researching RTP enhancements for increased packet reliability while keeping UDP speed

Projects

Kaboot - github.com/stran9682/kaboot *August 2025 – present*

- Real-time browser-based multiplayer quiz game with an unrestricted number of players per lobby
- Enabled live player status tracking using ASP.NET SignalR for real-time communication between client and server
- Stored lobby data and player sessions in Redis for server scalability, while also maintaining read and write speed
- Set up Kubernetes cluster in CloudLab to manage application microservices, providing reliability and availability
- Configured ingress to support web sockets along with routing traffic to correct backend deployment
- Implemented CI with GitHub actions, pushing only changed Docker images to Docker hub to reduce redundancy

VSCode R Code Review Tool Extension *January 2025 – May 2025*

- Code review tool for VSCode created to check for standards compliance of R code files using lexical analysis
- Developed modular architecture which utilizes local R installation concurrently using NodeJS child processes
- Used language server protocol to interact with VSCode, highlighting rule violations and issuing error messages

StyleMe - github.com/stran9682/StyleMe *January 2025 – May 2025*

- AI-powered fashion app using computer vision to provide user clothing recommendations based on body shape.
- Created segmentation mask with Google MediaPipe, then used NumPy to analyze user body features from mask
- Web scraped clothing items using Selenium, gathering data about price, fit, type, and color for recommendations.

- Handled user authentication with JWTs and CRUD database operations using ASP.NET Core web API
- Designed frontend to display catalog and outfit creation and implemented interactivity with React + TypeScript
- Containerized application and deployed to CloudLab with Docker Compose

Personal Website - github.com/stran9682/PersonalWebsite *October 2024 – January 2025*

- Personal website for sharing portfolio and contact information
- Used SQLite database to store image file locations and metadata then retrieve them with EFCore asynchronously.
- Implemented interactive Blazor components with server-side rendering and component routing.
- Set up, then deployed to production cloud environment with Azure App Service

Skills

- **Cloud and DevOps:** Git, GitHub Actions, Azure, Docker, Kubernetes
- **Frameworks & Tools:** React, ASP.NET Core, Flask, Selenium, SignalR, MatplotLib, EFCore
- **Languages**: JavaScript, Typescript, HTML, C, C++, Java, Python, C#, R, OCaml

Logan Yates

loganllyates@yahoo.com|(267)-905-6482|Norristown, PA|
https://www.linkedin.com/in/logan-yates-50771b256/

EDUCATION

West Chester University **West Chester, PA**

Bachelor's Degree in Computer Science *Graduation Date: May 2026*
GPA: 3.6

Minor in Music Production

Related Courses
Computer Science III, Data Structures and Algorithm,

CAMPUS INVOLVEMENT

Gospel Ministries

Treasurer Dec.
2023-Present

- Manage the funding and budget for the organization alongside with the secretary
- Develop monthly funding goals to help meet our organizational needs
- Purchasing assets that could supplement the engagement of other organization members

Computer Science Club Member

HTML Websites *Aug 2022 - Jun 2022*

- Learned JavaScript programming language to add interactive elements to websites, improving user
- experience.
- Utilized CSS along with JavaScript to enhance website design and layout.

Developed and implemented HTML coding techniques to create visually appealing websites.

Bowling Team Captain **Norristown, PA**

Bowling Competitions *Aug 2020 - Jun 2022*

- Taught new team members proper bowling techniques resulting in a improvement in their individual scores after one month of training.
- Learned how to encourage others to never give up and keep moving forward.
- Developed and executed a strategic game plan for each match, leading the team to win 80% of matches during the season.

Robotics Club Member **Norristown, PA**

Robotic Competition Games *Aug 2020 - Feb 2021*

- Learned how to build, plan, and organize a complex robot project, collaborating with group members and contributing unique ideas to enhance the robot's functionality.
- Utilized V8 (Vex Code) Software to program the robot's movements and actions, resulting in improved efficiency and performance.

Volunteer Work

Early Move-in ***West Chester, PA***

Volunteer *Fall 2024*

- Assisted in helping incoming students settle into their living spaces on the campus

SKILLS & INTERESTS

Skills: Hard Worker, Communication, Leadership, Problem Solver, Creativity, Teamwork, Time Management

Interest: Working Out, Physical Activities, Music Production, Computer Programming, Cooking

ACHIVEMENTS

Deans Lists (Fall 2022, Spring 2023)

Ali Abdulrazzak
Philadelphia PA, 19124
(484)-773-6259
AA1032220@wcupa.edu

Education:West Chester University — West Chester, PA
B.S. in Computer Science ~ Expected May 2026
Lehigh Carbon Community College - Allentown, PA
A.S. in Computer Science - Graduated May 2023

Skills

- Microsoft Office Suite
- Java Programming and C++ Programming
- SQL
- Computer Security and Networking
- IT solutions

Relevant Coursework

- Computer Programming in C++ and Java
- Data Structure and Algorithm
- Computer Security and Networking
- Operating Systems

Relevant Experience

Shift4— Allentown, PA
POS technician — August 2024 - January 2025

- Configured, and maintained Shift4 point-of-sale (POS) systems for retail and hospitality clients, ensuring seamless integration with payment processing platforms.
- Diagnosed hardware and software issues on-site and remotely, minimizing downtime and improving operational efficiency.
- Performed system updates, security patches, and backup procedures in compliance with PCI-DSS standards.
- Provided technical support and issue resolution for Shift4 terminals, network connectivity, and integrated devices (e.g., receipt printers, barcode scanners).

- Collaborated with internal IT, sales, and customer service teams.

- Documented service tickets, system configurations, and customer interactions using CRM and ticketing platforms.

Brosnan Security— Philadelphia, PA

Security Guard— April 2021 – June 2024

- Completed shift reports, documenting all security-related activity. Used computers to aid.

ACTIVITIES: Computer Programming Club, Competitive Programming Club, Game Development Club